

Informe de Laboratorio 08

Tema: Compresión

Nota

Estudiante	Escuela	Asignatura
Sebastian Arley Chirinos Negrón /Rodrigo Estefano Viza Cuti schirinosne@unsa.edu.pe / rvizac@unsa.edu.pe	Escuela Profesional de Ingeniería de Sistemas	Estructura de datos y Algoritmos Semestre: I Código: 1702124

Laboratorio	Tema	Duración
08	Compresión	04 horas

Semestre académico	Fecha de inicio	Fecha de entrega
2023 - A	24 Julio 2023	31 Julio 2023

1. Tarea

1.1. Compresión

- La compresión de texto con el algoritmo de Huffman es una técnica utilizada para reducir el tamaño de archivos de texto al codificarlos de manera eficiente. Este algoritmo fue propuesto por David A. Huffman en 1952 y se basa en la idea de asignar códigos más cortos a los caracteres que aparecen con mayor frecuencia en el texto y códigos más largos a los caracteres menos frecuentes.
- El proceso de compresión con el algoritmo de Huffman implica los siguientes pasos:
 - Análisis de frecuencias: Se analiza el texto para determinar la frecuencia de aparición de cada carácter. Los caracteres más frecuentes recibirán códigos más cortos, y los menos frecuentes recibirán códigos más largos.
 - Construcción del árbol de Huffman: Se crea un árbol binario donde los nodos hoja representan los caracteres y los nodos internos son la suma de las frecuencias de sus nodos hijos. Este árbol se construye de manera que los caracteres más frecuentes estén más cerca de la raíz y los menos frecuentes estén en las hojas más alejadas.
 - Asignación de códigos: Recorriendo el árbol de Huffman, se asignan códigos binarios a cada carácter, tomando un camino a la izquierda cuando se encuentra un "0" o un camino a la derecha cuando se encuentra un "1".
 - Codificación del texto: Finalmente, el texto original se reemplaza por su representación en forma de secuencia de códigos binarios, utilizando la asignación de códigos realizada en el paso anterior.

- La compresión con el algoritmo de Huffman es eficiente para reducir el tamaño de archivos de texto, especialmente cuando hay caracteres que aparecen con mucha frecuencia. Los caracteres más comunes tendrán códigos cortos, lo que ayuda a lograr una mayor compresión en comparación con codificaciones fijas donde todos los caracteres tienen la misma longitud de código.
- Al ser un algoritmo sin pérdida, la compresión con Huffman permite recuperar el texto original sin ninguna pérdida de información. La descompresión implica recorrer el árbol de Huffman utilizando la secuencia de códigos binarios y reemplazando los códigos con los caracteres correspondientes hasta que se obtiene el texto original.
- Es importante tener en cuenta que la compresión con el algoritmo de Huffman es más efectiva en textos largos con patrones repetitivos o caracteres que se repiten con frecuencia. En textos muy cortos o con una distribución de caracteres uniforme, la compresión puede no ser significativa o incluso puede aumentar el tamaño del archivo.
- La compresión de texto con el algoritmo de Huffman es ampliamente utilizado en aplicaciones donde el espacio de almacenamiento o el ancho de banda son limitados, como en la compresión de archivos, transmisión de datos en redes, y otras aplicaciones donde se requiere una reducción del tamaño del texto sin perder información.

2. URL de Repositorio Github

- URL del Repositorio GitHub para clonar o recuperar.
- <https://github.com/Dan-Ar5/eda-lab-b-23a-b>

3. Ejercicios designados

3.1. Clase Nodo

Listing 1: NodoHuff.java

```
1 public class NodoHuff implements Comparable<NodoHuff>{
2     private final int frequency;
3
4     private int height = 1;
5
6     public void setData(String data) {
7         this.data = data;
8     }
9
10    private String data = "";
11    private NodoHuff left;
12
13    private NodoHuff right;
14
15    public NodoHuff(int freq) {
16        this.frequency = freq;
17    }
18    public NodoHuff(NodoHuff left, NodoHuff right) {
19        this.frequency = left.getFrequency() + right.getFrequency();
20        this.left = left;
21        this.right = right;
22        this.data = String.valueOf(frequency);
23    }
```

```
23 }
24
25
26 public int getHeight() {
27     return height;
28 }
29 public void setHeight(int height) {
30     this.height = height;
31 }
32 public String getData() {
33     return data;
34 }
35
36 public NodoHuff getRight() {
37     return right;
38 }
39
40 public NodoHuff getLeft() {
41     return left;
42 }
43
44
45 // ya que este nodo no almacena directamente caracteres tenemos que crear una clase
46 // hija, porque solo las hojas son las que almacenan caracteres
47 @Override
48 public int compareTo(NodoHuff nodoHuff) {
49     return Integer.compare(frequency, nodoHuff.getFrequency());
50 }
51
52 public int getFrecuency() {
53     return frequency;
54 }
55
56 public int getFrequency() {
57     return frequency;
58 }
59 }
```

- Línea 1-5: Se define la clase 'NodoHuff' que implementa la interfaz 'Comparable<NodoHuff>'. La clase tiene los siguientes atributos y métodos: - 'private final int frequency;': Atributo privado y final que almacena la frecuencia (cantidad de apariciones) del nodo. - 'private int height = 1;': Atributo privado que representa la altura del nodo en el árbol de Huffman, inicializado en 1. - 'public void setData(String data) this.data = data;': Método público que establece el contenido del nodo a partir de un dato de tipo 'String'. - 'private String data = "";': Atributo privado que almacena el contenido del nodo. Se inicializa como una cadena vacía. - 'private NodoHuff left;': Atributo privado de tipo 'NodoHuff' que representa el hijo izquierdo del nodo. - 'private NodoHuff right;': Atributo privado de tipo 'NodoHuff' que representa el hijo derecho del nodo.
- Línea 7-11: Se define el constructor de la clase 'NodoHuff' que recibe un parámetro 'freq' de tipo 'int'. Este constructor se utiliza para crear un nodo hoja con la frecuencia especificada. La frecuencia se almacena en el atributo 'frequency'.
- Línea 12-20: Se define otro constructor de la clase 'NodoHuff' que recibe dos parámetros 'left' y 'right', ambos de tipo 'NodoHuff'. Este constructor se utiliza para crear un nodo interno que tiene dos hijos. Calcula su frecuencia como la suma de las frecuencias de sus hijos y la almacena

en el atributo 'frequency'. El contenido del nodo se establece como la representación en cadena de la frecuencia.

- Línea 22-26: Se define el método público 'getHeight()' que devuelve la altura del nodo en el árbol de Huffman.
- Línea 27-31: Se define el método público 'setHeight()' que permite establecer la altura del nodo en el árbol de Huffman.
- Línea 33-37: Se define el método público 'getData()' que devuelve el contenido del nodo (carácter o frecuencia).
- Línea 39-43: Se define el método público 'getRight()' que devuelve el hijo derecho del nodo.
- Línea 45-49: Se define el método público 'getLeft()' que devuelve el hijo izquierdo del nodo.
- Línea 52-58: Se sobrescribe el método 'compareTo()' de la interfaz 'Comparable'. Este método se utiliza para comparar dos nodos en función de su frecuencia. Devuelve un valor negativo si la frecuencia del nodo actual es menor, cero si son iguales, y un valor positivo si es mayor.
- Línea 60-64: Se define el método público 'getFrequency()' que devuelve la frecuencia del nodo.
- Línea 66-70: Se define el método público 'getFrequency()' que devuelve la frecuencia del nodo. Este método es exactamente igual al anterior 'getFrequency()'.

3.2. Clase Hoja

Listing 2: Leaf.java

```
1 public class Leaf extends NodoHuff {  
2     private final char character;  
3  
4     private String data;  
5     public char getCaracter() {  
6         return character;  
7     }  
8     public Leaf(char character, int freq) {  
9         super(freq);  
10        this.character = character;  
11        this.data = this.character + " " + this.getFrequency();  
12    }  
13    public String getData() {  
14        return data;  
15    }  
16 }
```

- Línea 1-6: Se define la clase 'Leaf', que extiende de la clase 'NodoHuff', lo que significa que 'Leaf' es una subclase de 'NodoHuff'. Esto indica que 'Leaf' hereda todos los atributos y métodos de la clase 'NodoHuff'.
- Línea 8-10: Se declara un atributo 'character' de tipo 'char', que representa el carácter almacenado en este nodo hoja.
- Línea 12-14: Se declara un atributo 'data' de tipo 'String', que almacenará una representación de la frecuencia y el carácter en este nodo hoja.

- Línea 16-18: Se define un método 'getCaracter()' que devuelve el carácter almacenado en este nodo hoja.
- Línea 20-27: Se define el constructor de la clase 'Leaf', que recibe como parámetros un carácter 'caracter' y una frecuencia 'freq'. El constructor utiliza 'super(freq)' para llamar al constructor de la clase padre 'NodoHuff' y asignar la frecuencia al nodo hoja. Luego, se asigna el carácter y se construye la cadena 'data' que contiene el carácter y su frecuencia.
- Línea 29-31: Se define el método 'getData()', que devuelve la cadena 'data' que contiene el carácter y su frecuencia.

3.3. Clase HuffmanTree.java

Listing 3: HuffmanTree.java

```
1 import java.util.HashMap;
2 import java.util.Map;
3 import java.util.PriorityQueue;
4 import java.util.Queue;
5
6 public class HuffmanTree {
7     private NodoHuff root;
8
9     private final String text;
10    private Map<Character, Integer> charFreq;
11    private final Map<Character, String> huffmanCod;
12    public HuffmanTree(String text) {
13        this.text = text;
14        fillCharFreqMap();
15        this.huffmanCod = new HashMap<>();
16    }
17
18    public void fillCharFreqMap() {
19        charFreq = new HashMap<>();
20        for (char character : text.toCharArray()) {
21            Integer integer = charFreq.get(character);
22            charFreq.put(character, integer != null ? integer + 1 : 1);
23        }
24    }
25
26    public NodoHuff getRoot() {
27        return root;
28    }
29
30    public String encode() {
31        Queue<NodoHuff> cola = new PriorityQueue<>();
32        charFreq.forEach((character, freq) -> cola.add(new Leaf(character, freq)));
33        while (cola.size() > 1) {
34            cola.add(new NodoHuff(col.poll(), col.poll()));
35        }
36        generateHuffmanCodes(root = cola.poll(), "");
37        return getEncodeText();
38    }
39
40    private void generateHuffmanCodes(NodoHuff nodoHuff, String code) {
41        if (nodoHuff instanceof Leaf) {
```

```

42         huffmanCod.put(((Leaf) nodoHuff).getCaracter(), code);
43         return;
44     }
45     // si nos vamos por la izquierda concatenamos con 0
46     generateHuffmanCodes(nodoHuff.getLeft(), code.concat("0"));
47     // si nos vamos por la derecha concatenamos con 1
48     generateHuffmanCodes(nodoHuff.getRight(), code.concat("1"));
49     updateHeight(nodoHuff);
50 }
51
52 private String getEncodeText() {
53     StringBuilder sb = new StringBuilder();
54     for (char caracter : text.toCharArray()) {
55         sb.append(huffmanCod.get(caracter));
56     }
57     return sb.toString();
58 }
59
60 public String decode(String encodeText) {
61     StringBuilder sb = new StringBuilder();
62     NodoHuff current = root;
63     for (char caracter : encodeText.toCharArray()) {
64         current = caracter == '0' ? current.getLeft() : current.getRight();
65         if (current instanceof Leaf) {
66             sb.append(((Leaf) current).getCaracter());
67             current = root;
68         }
69     }
70     return sb.toString();
71 }
72 public void updateHeight(NodoHuff current) {
73     int maxHeight = Math.max(
74         height(current.getLeft()),
75         height(current.getRight())
76     );
77     current.setHeight(maxHeight + 1);
78 }
79
80 public int height(NodoHuff nodoHuff) {
81     return (nodoHuff != null) ? nodoHuff.getHeight() : 0;
82 }
83
84 // metodo para imprimir el los caracteres con sus respectivos codigos huffman
85 public void printCodes () {
86     huffmanCod.forEach((caracter, freq) -> System.out.println(caracter + ": " + freq)) ;
87 }
88 }

```

- Línea 1-7: Se importan las clases necesarias para el funcionamiento del programa.
- Línea 9-23: Se define la clase 'HuffmanTree', que representa el árbol de Huffman. Esta clase contiene varios atributos, incluyendo el nodo raíz del árbol ('root'), el texto que se desea comprimir ('text'), un mapa que almacena la frecuencia de cada carácter en el texto ('charFreq'), y un mapa que almacena los códigos Huffman de cada carácter ('huffmanCod'). El constructor de la clase recibe el texto que se desea comprimir y llama al método 'fillCharFreqMap()' para calcular las frecuencias de los caracteres en el texto.

- Línea 25-35: Se define el método ‘fillCharFreqMap()’, que calcula la frecuencia de cada carácter en el texto y los almacena en el mapa ‘charFreq’.
- Línea 37-41: Se define el método ‘getRoot()’, que devuelve el nodo raíz del árbol de Huffman.
- Línea 43-63: Se define el método ‘encode()’, que realiza la codificación del texto utilizando el algoritmo de Huffman. Primero, se crea una cola de prioridad (‘cola’) que contiene nodos hoja para cada carácter y su frecuencia. Luego, se utiliza la cola de prioridad para construir el árbol de Huffman y se generan los códigos Huffman para cada carácter. Finalmente, se devuelve el texto codificado.
- Línea 65-82: Se define el método ‘generateHuffmanCodes()’, que genera los códigos Huffman para cada carácter en el árbol. Este método es recursivo y utiliza un código binario (‘code’) para representar el camino hacia cada nodo hoja. Los códigos Huffman se almacenan en el mapa ‘huffmanCod’.
- Línea 84-92: Se define el método ‘getEncodeText()’, que devuelve el texto codificado a partir de los códigos Huffman generados para cada carácter en el texto original.
- Línea 94-119: Se define el método ‘decode()’, que realiza la decodificación del texto codificado utilizando el árbol de Huffman. El método recibe el texto codificado y utiliza el árbol para seguir el camino de cada bit (‘0’ o ‘1’) y obtener el carácter original. El texto decodificado se almacena en un ‘StringBuilder’ y se devuelve al final del método.
- Línea 121-135: Se definen los métodos ‘updateHeight()’ y ‘height()’, que se utilizan para actualizar y obtener la altura de un nodo en el árbol de Huffman, respectivamente.
- Línea 137-147: Se define el método ‘printCodes()’, que imprime los caracteres junto con sus respectivos códigos Huffman en la consola.

En resumen, el código implementa el algoritmo de compresión de Huffman para codificar y decodificar textos utilizando un árbol de Huffman. El árbol de Huffman se construye a partir de las frecuencias de los caracteres en el texto y se utiliza para generar los códigos Huffman para cada carácter. La codificación y decodificación del texto se realizan siguiendo el camino de cada carácter en el árbol de Huffman.

3.4. APP TextCompressor.java

Listing 4: TextCompressor.java

```
1 import org.graphstream.graph.Edge;
2 import org.graphstream.graph.Graph;
3 import org.graphstream.graph.IdAlreadyInUseException;
4 import org.graphstream.graph.Node;
5 import org.graphstream.graph.implementations.SingleGraph;
6 import org.graphstream.ui.view.Viewer;
7 import com.formdev.flatlaf.intellijthemes.materialthemeuilite.FlatMonokaiProContrastIJTheme;
8 import javax.swing.JOptionPane;
9
10 public class TextCompressor extends javax.swing.JFrame {
11
12     /**
13      * Creates new form HuffmanApp
14      */
15     public TextCompressor() {
16         initComponents();
```

```
17     setLocationRelativeTo(null);
18     setTitle("Huffman Encoder");
19 }
20
21 /**
22  * This method is called from within the constructor to initialize the form.
23  * WARNING: Do NOT modify this code. The content of this method is always
24  * regenerated by the Form Editor.
25  */
26 @SuppressWarnings("unchecked")
27 // <editor-fold defaultstate="collapsed" desc="Generated Code">
28 private void initComponents() {
29
30     jPanel2 = new javax.swing.JPanel();
31     btEncode = new javax.swing.JButton();
32     jScrollPane2 = new javax.swing.JScrollPane();
33     jTextArea2 = new javax.swing.JTextArea();
34     jLabel5 = new javax.swing.JLabel();
35     jLabel4 = new javax.swing.JLabel();
36     jScrollPane4 = new javax.swing.JScrollPane();
37     taDescomprimido = new javax.swing.JTextPane();
38     jLabel2 = new javax.swing.JLabel();
39     jScrollPane3 = new javax.swing.JScrollPane();
40     taComp = new javax.swing.JTextPane();
41     jScrollPane1 = new javax.swing.JScrollPane();
42     txPlano = new javax.swing.JTextArea();
43     jLabel3 = new javax.swing.JLabel();
44     btDecode = new javax.swing.JButton();
45     jLabel1 = new javax.swing.JLabel();
46     jLabel8 = new javax.swing.JLabel();
47
48     setDefaultCloseOperation(javax.swing.WindowConstants.EXIT_ON_CLOSE);
49
50     btEncode.setText(" COMPRIMIR");
51     btEncode.addMouseListener(new java.awt.event.MouseAdapter() {
52         public void mouseClicked(java.awt.event.MouseEvent evt) {
53             btEncodeMouseClicked(evt);
54         }
55     });
56     btEncode.addActionListener(new java.awt.event.ActionListener() {
57         public void actionPerformed(java.awt.event.ActionEvent evt) {
58             btEncodeActionPerformed(evt);
59         }
60     });
61
62     jTextArea2.setColumns(20);
63     jTextArea2.setRows(5);
64     jScrollPane2.setViewportView(jTextArea2);
65
66     jLabel5.setFont(new java.awt.Font("Roboto", 0, 12)); // NOI18N
67     jLabel5.setText("Texto a descomprimir: ");
68
69     jLabel4.setFont(new java.awt.Font("Roboto", 0, 12)); // NOI18N
70     jLabel4.setText("Texto descomprimido:");
71
72     jScrollPane4.setViewportView(taDescomprimido);
```

```
73
74     jLabel2.setFont(new java.awt.Font("Roboto", 0, 12)); // NOI18N
75     jLabel2.setText("Texto comprimido:");
76
77     jScrollPane3.setViewportViewView(taComp);
78
79     txPlano.setColumns(20);
80     txPlano.setRows(5);
81     jScrollPane1.setViewportViewView(txPlano);
82
83     jLabel3.setFont(new java.awt.Font("Roboto", 0, 12)); // NOI18N
84     jLabel3.setText("Texto a comprimir: ");
85
86     btDecode.setText("DESCOMPRIMIR");
87     btDecode.addMouseListener(new java.awt.event.MouseAdapter() {
88         public void mouseClicked(java.awt.event.MouseEvent evt) {
89             btDecodeMouseClicked(evt);
90         }
91     });
92     btDecode.addActionListener(new java.awt.event.ActionListener() {
93         public void actionPerformed(java.awt.event.ActionEvent evt) {
94             btDecodeActionPerformed(evt);
95         }
96     });
97
98     jLabel1.setFont(new java.awt.Font("Verdana", 1, 24)); // NOI18N
99     jLabel1.setHorizontalAlignment(javax.swing.SwingConstants.CENTER);
100    jLabel1.setText("TEXT");
101
102    jLabel8.setFont(new java.awt.Font("Verdana", 1, 24)); // NOI18N
103    jLabel8.setHorizontalAlignment(javax.swing.SwingConstants.CENTER);
104    jLabel8.setText("COMPRESSOR");
105
106    javax.swing.GroupLayout jPanel2Layout = new javax.swing.GroupLayout(jPanel2);
107    jPanel2.setLayout(jPanel2Layout);
108    jPanel2Layout.setHorizontalGroup(
109        jPanel2Layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
110            .addGroup(jPanel2Layout.createSequentialGroup()
111                .addGap(125, 125, 125)
112                .addComponent(jLabel1)
113                .addGap(18, 18, 18)
114                .addComponent(jLabel8)
115                .addGap(javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)
116            )
117            .addGroup(jPanel2Layout.createSequentialGroup()
118                .addGap(59, 59, 59)
119                .addGroup(jPanel2Layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
120                    .addComponent(jScrollPane3,
121                        javax.swing.GroupLayout.Alignment.TRAILING,
122                        javax.swing.GroupLayout.PREFERRED_SIZE, 269,
123                        javax.swing.GroupLayout.PREFERRED_SIZE)
124                    .addGroup(jPanel2Layout.createSequentialGroup()
125                        .addComponent(jLabel3,
126                            javax.swing.GroupLayout.PREFERRED_SIZE, 114,
127                            javax.swing.GroupLayout.PREFERRED_SIZE)
```

```

122         .addGap(154, 154, 154))
123     .addGroup(javax.swing.GroupLayout.Alignment.TRAILING,
        jPanel2Layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING,
        false)
124     .addGroup(javax.swing.GroupLayout.Alignment.TRAILING,
        jPanel2Layout.createSequentialGroup()
125         .addComponent(btEncode)
126         .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED,
            javax.swing.GroupLayout.DEFAULT_SIZE,
            Short.MAX_VALUE)
127         .addGroup(jPanel2Layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING,
            false)
128             .addComponent(jLabel12)
129             .addComponent(jScrollPane1,
                javax.swing.GroupLayout.PREFERRED_SIZE,
                268,
                javax.swing.GroupLayout.PREFERRED_SIZE)))
130     .addGroup(javax.swing.GroupLayout.Alignment.TRAILING,
        jPanel2Layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING,
        false)
131         .addComponent(jLabel4)
132         .addComponent(jLabel5)
133         .addComponent(jScrollPane4,
            javax.swing.GroupLayout.PREFERRED_SIZE,
            269,
            javax.swing.GroupLayout.PREFERRED_SIZE)
134         .addGroup(jPanel2Layout.createSequentialGroup()
135             .addComponent(jScrollPane2,
                javax.swing.GroupLayout.PREFERRED_SIZE,
                268,
                javax.swing.GroupLayout.PREFERRED_SIZE)
136             .addGap(18, 18, 18)
137             .addComponent(btDecode))))
138     .addContainerGap())
139 );
140 jPanel2Layout.setVerticalGroup(
141     jPanel2Layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
142     .addGroup(jPanel2Layout.createSequentialGroup()
143         .addGap(22, 22, 22)
144         .addGroup(jPanel2Layout.createParallelGroup(javax.swing.GroupLayout.Alignment.BASELINE)
145             .addComponent(jLabel1,
                javax.swing.GroupLayout.PREFERRED_SIZE, 31,
                javax.swing.GroupLayout.PREFERRED_SIZE)
146             .addComponent(jLabel8,
                javax.swing.GroupLayout.PREFERRED_SIZE, 31,
                javax.swing.GroupLayout.PREFERRED_SIZE))
147         .addGap(32, 32, 32)
148         .addComponent(jLabel3)
149         .addGap(18, 18, 18)
150         .addGroup(jPanel2Layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING,
            false)
151             .addComponent(jScrollPane1,
                javax.swing.GroupLayout.PREFERRED_SIZE, 44,
                javax.swing.GroupLayout.PREFERRED_SIZE)
152             .addComponent(btEncode))
153         .addGap(18, 18, 18)
154         .addComponent(jLabel2)
155         .addGap(18, 18, 18)

```

```
156         .addComponent(jScrollPane3,  
157             javax.swing.GroupLayout.PREFERRED_SIZE, 45,  
158             javax.swing.GroupLayout.PREFERRED_SIZE)  
159         .addGap(23, 23, 23)  
160         .addComponent(jLabel15)  
161         .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED)  
162         .addGroup(jPanel2Layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)  
163             .addComponent(jScrollPane2,  
164                 javax.swing.GroupLayout.PREFERRED_SIZE, 56,  
165                 javax.swing.GroupLayout.PREFERRED_SIZE)  
166             .addComponent(btDecode))  
167         .addGap(18, 18, 18)  
168         .addComponent(jLabel14)  
169         .addGap(13, 13, 13)  
170         .addComponent(jScrollPane4,  
171             javax.swing.GroupLayout.PREFERRED_SIZE, 56,  
172             javax.swing.GroupLayout.PREFERRED_SIZE)  
173         .addContainerGap(56, Short.MAX_VALUE))  
174     );  
175  
176     javax.swing.GroupLayout layout = new javax.swing.GroupLayout(getContentPane());  
177     getContentPane().setLayout(layout);  
178     layout.setHorizontalGroup(  
179         layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)  
180             .addGroup(javax.swing.GroupLayout.Alignment.TRAILING,  
181                 layout.createSequentialGroup()  
182                     .addComponent(jPanel2, javax.swing.GroupLayout.PREFERRED_SIZE,  
183                         javax.swing.GroupLayout.DEFAULT_SIZE,  
184                         javax.swing.GroupLayout.PREFERRED_SIZE)  
185                     .addContainerGap(39, Short.MAX_VALUE))  
186     );  
187     layout.setVerticalGroup(  
188         layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)  
189             .addGroup(javax.swing.GroupLayout.Alignment.TRAILING,  
190                 layout.createSequentialGroup()  
191                     .addContainerGap()  
192                     .addComponent(jPanel2, javax.swing.GroupLayout.DEFAULT_SIZE,  
193                         javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE))  
194     );  
195  
196     pack();  
197 }  
198  
199 private void btDecodeMouseClicked(java.awt.event.MouseEvent evt) {  
200     if(txPlano.getText().equals("")) {  
        JOptionPane.showMessageDialog(null, "Primero codifique el mensaje");  
    }  
    else {  
        HuffmanTree arbol = new HuffmanTree(txPlano.getText());  
        String codificado = arbol.encode();  
        String decodificado = arbol.decode(codificado);  
        taDescomprimido.setText(decodificado);  
        crearGrafo(txPlano.getText());  
    }  
}
```

```
201 private void btEncodeMouseClicked(java.awt.event.MouseEvent evt) {
202     HuffmanTree arbol = new HuffmanTree(txPlano.getText());
203     String codificado = arbol.encode();
204     taComp.setText(codificado);
205     crearGrafo(txPlano.getText());
206 }
207
208 private void btEncodeActionPerformed(java.awt.event.ActionEvent evt) {
209     // TODO add your handling code here:
210 }
211
212 private void btDecodeActionPerformed(java.awt.event.ActionEvent evt) {
213     // TODO add your handling code here:
214 }
215
216 /**
217  * @param args the command line arguments
218  */
219 public static void main(String args[]) {
220     /* Set the Nimbus look and feel */
221     <editor-fold defaultstate="collapsed" desc=" Look and feel setting code (optional)
222     ">
223     /* If Nimbus (introduced in Java SE 6) is not available, stay with the default look
224     and feel.
225     * For details see
226     http://download.oracle.com/javase/tutorial/uiswing/lookandfeel/plaf.html
227     */
228     try {
229         for (javax.swing.UIManager.LookAndFeelInfo info :
230             javax.swing.UIManager.getInstalledLookAndFeels()) {
231             if ("Nimbus".equals(info.getName())) {
232                 javax.swing.UIManager.setLookAndFeel(info.getClassName());
233                 break;
234             }
235         }
236     } catch (ClassNotFoundException ex) {
237
238         java.util.logging.Logger.getLogger(HuffmanApp.class.getName()).log(java.util.logging.Level.SEVERE,
239         null, ex);
240     } catch (InstantiationException ex) {
241
242         java.util.logging.Logger.getLogger(HuffmanApp.class.getName()).log(java.util.logging.Level.SEVERE,
243         null, ex);
244     } catch (IllegalAccessException ex) {
245
246         java.util.logging.Logger.getLogger(HuffmanApp.class.getName()).log(java.util.logging.Level.SEVERE,
247         null, ex);
248     } catch (javax.swing.UnsupportedLookAndFeelException ex) {
249
250         java.util.logging.Logger.getLogger(HuffmanApp.class.getName()).log(java.util.logging.Level.SEVERE,
251         null, ex);
252     }
253 }
254 </editor-fold>
255 FlatMonokaiProContrastIJTheme.setup();
256 /* Create and display the form */
257 java.awt.EventQueue.invokeLater(() -> {
```

```
245         new TextCompressor().setVisible(true);
246     });
247
248
249
250     }
251
252     // Variables declaration - do not modify
253     private javax.swing.JButton btDecode;
254     private javax.swing.JButton btEncode;
255     private javax.swing.JLabel jLabel1;
256     private javax.swing.JLabel jLabel2;
257     private javax.swing.JLabel jLabel3;
258     private javax.swing.JLabel jLabel4;
259     private javax.swing.JLabel jLabel5;
260     private javax.swing.JLabel jLabel8;
261     private javax.swing.JPanel jPanel2;
262     private javax.swing.JScrollPane jScrollPane1;
263     private javax.swing.JScrollPane jScrollPane2;
264     private javax.swing.JScrollPane jScrollPane3;
265     private javax.swing.JScrollPane jScrollPane4;
266     private javax.swing.JTextArea jTextArea2;
267     private javax.swing.JTextPane taComp;
268     private javax.swing.JTextPane taDescomprimido;
269     private javax.swing.JTextArea txPlano;
270     // End of variables declaration
271     public static void crearGrafo(String text) {
272         HuffmanTree arbol = new HuffmanTree(text);
273         arbol.encode();
274         System.setProperty("org.graphstream.ui", "swing");
275         // Crear un grafo
276         Graph graph = new SingleGraph("Aplicacion");
277         // Establecer la hoja de estilo para la representacin visual de los nodos y las
278         aristas
279         graph.setAttribute("ui.stylesheet", styleSheet);
280
281         // Para visualizar el grafo en una ventana con tamao personalizado
282         addNodesToGraph(arbol.getRoot(), graph, 0, 0);
283         Viewer viewer = graph.display();
284         viewer.disableAutoLayout();
285     }
286     private static void addNodesToGraph(NodoHuff current, Graph graph, double x, double y) {
287         if (current != null) {
288             try { // Agregar el nodo actual al grafo
289                 Node node = graph.addNode(String.valueOf(current.getData()));
290                 node.setAttribute("ui.label", current.getData());
291                 node.setAttribute("xy", x, y);
292                 double leftX = x - 10 * Math.pow(2, current.getHeight());
293                 double rightX = x + 10 * Math.pow(2, current.getHeight());
294                 double childY = y - 100;
295
296                 // Agregar los nodos hijos recursivamente
297                 if (current.getLeft() != null) {
298                     addNodesToGraph(current.getLeft(), graph, leftX, childY);
```

```

299         Edge edge = graph.addEdge((current.getData() + "-" +
300             current.getLeft().getData()),
301             String.valueOf(current.getData()),
302             String.valueOf(current.getLeft().getData()));
303         edge.setAttribute("ui.label", "0");
304     }
305     if (current.getRight() != null) {
306         addNodesToGraph(current.getRight(), graph, rightX, childY);
307         Edge edge = graph.addEdge((current.getData() + "-" +
308             current.getRight().getData()),
309             String.valueOf(current.getData()),
310             String.valueOf(current.getRight().getData()));
311         edge.setAttribute("ui.label", "1");
312     }
313 }
314 }
315 }
316 protected static String styleSheet = "node {" +
317     "    shape: box;" +
318     "    size: 40px;" +
319     "    text-size: 12;" +
320     "    fill-mode: plain;" +
321     "    fill-color: yellow;" +
322     "    stroke-mode: plain;" +
323     "    stroke-color: black;" +
324     "    stroke-width: 1px;" +
325     "}" +
326     "edge {" +
327     "    arrow-shape: arrow;" +
328     "    arrow-size: 10px, 4px;" +
329     "    text-size: 15;" +
330     "    text-color: black;" +
331     "}" +
332     "node.root {" +
333     "    fill-color: yellow;" +
334     "}" +
335 }
```

- Línea 1-18: Se importan las clases necesarias para el funcionamiento del programa, incluyendo las relacionadas con la interfaz gráfica (swing) y las de GraphStream para visualizar el árbol de Huffman.
- Línea 20-24: Se define la clase 'TextCompressor', que extiende la clase 'javax.swing.JFrame' para crear una ventana de interfaz gráfica para la compresión y descompresión de texto. El constructor de la clase inicializa la interfaz y establece el título de la ventana.
- Línea 27-271: Se define el código generado automáticamente para inicializar la interfaz gráfica. Este código crea los componentes visuales, como botones, áreas de texto y etiquetas, y los posiciona en la ventana.
- Línea 38-59: Se definen los métodos 'btDecodeMouseClicked()' y 'btEncodeMouseClicked()', que se ejecutan cuando se hace clic en los botones "DESCOMPRIMIR" y "COMPRIMIR" respectivamente.

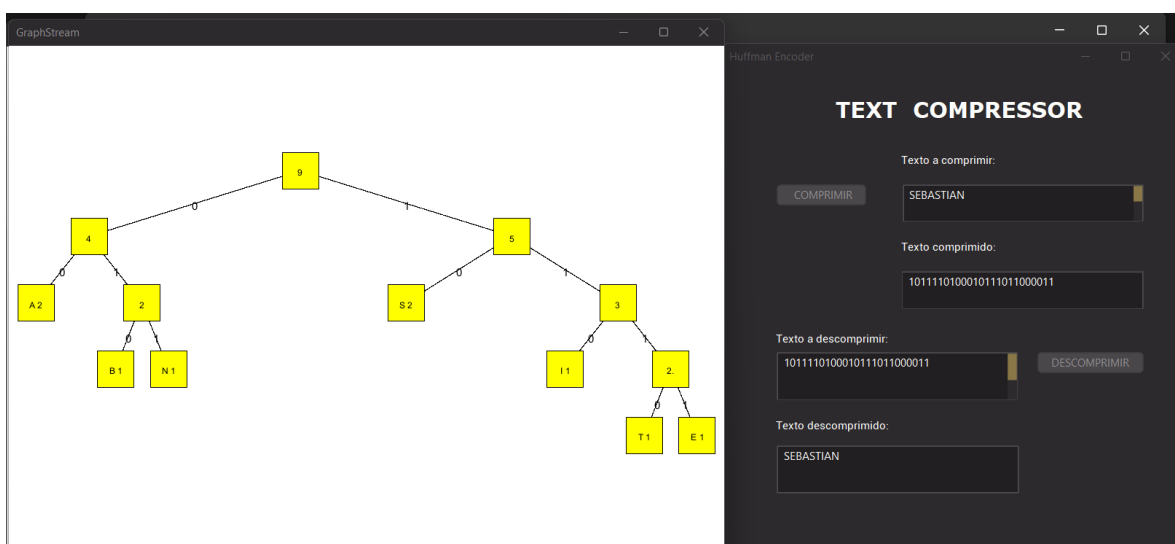
te. Estos métodos se encargan de realizar la compresión y descompresión del texto utilizando la clase 'HuffmanTree'. También se crea un grafo para visualizar el árbol de Huffman utilizando GraphStream.

- Línea 65-70: Se define el método 'main()', que es el punto de entrada del programa. En este método se crea una instancia de la clase 'TextCompressor' y se muestra la ventana de la interfaz gráfica.
- Línea 73-128: Se define el método 'crearGrafo()', que crea y muestra un grafo utilizando GraphStream para visualizar el árbol de Huffman. El método recibe el texto que se va a comprimir, crea una instancia de la clase 'HuffmanTree', y utiliza los métodos de esta clase para generar el árbol de Huffman y codificar el texto. Luego, crea el grafo y agrega los nodos y aristas correspondientes a cada nodo del árbol de Huffman.
- Línea 130-166: Se define el método 'addNodesToGraph()', que es un método recursivo utilizado por 'crearGrafo()' para agregar los nodos y aristas al grafo. Este método recibe el nodo actual del árbol de Huffman ('current'), el grafo en el que se agregarán los nodos y aristas ('graph'), y las coordenadas ('x' e 'y') para la posición del nodo en la visualización del grafo. El método agrega el nodo actual al grafo y luego agrega los nodos hijos recursivamente, conectándolos con aristas etiquetadas con "0" y "1" para indicar el camino hacia cada hijo.
- Línea 169-182: Se define una hoja de estilo ('styleSheet') para la representación visual de los nodos y aristas en el grafo de Huffman. Especifica el tamaño y el color de los nodos, así como el tamaño y el color de las etiquetas de las aristas.

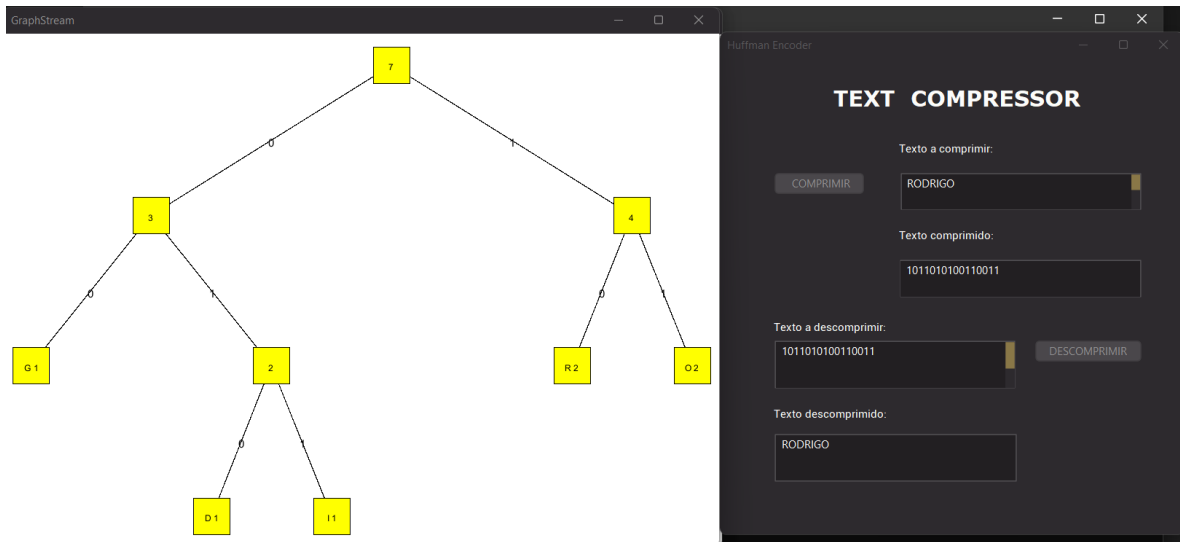
En resumen, el código implementa una interfaz gráfica ('TextCompressor') para comprimir y descomprimir texto utilizando el algoritmo de Huffman. La interfaz muestra campos de texto para ingresar el texto a comprimir y mostrar el texto comprimido y descomprimido. También hay botones para realizar las operaciones de compresión y descompresión. Además, utiliza la biblioteca GraphStream para visualizar el árbol de Huffman generado durante el proceso de compresión y descompresión.

3.5. Ejecución

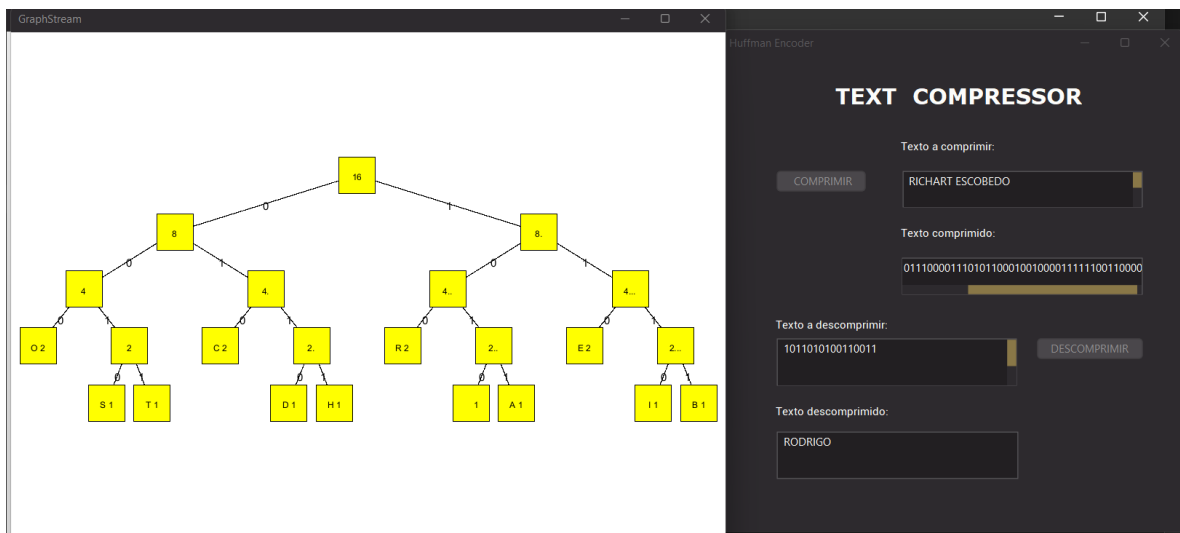
■ EJECUCION 1



■ EJECUCION 2



■ EJECUCION 3



3.6. Commits del trabajo

- Acá estan algunos de los commits mas importantes en este trabajo:

```
PowerShell 7.3.6
PS C:\Users\Sewas\Desktop\LABS\EDA-LAB-B-23A> git log
commit 3d75087c69b2698e6ea9afb4537051d81f4e67fd (HEAD)
Author: BastleyNait <schirinosne@unsa.edu.pe>
Date: Sun Jul 30 21:07:38 2023 -0500

    implementacion graph stream

commit 42e5d60a4a1adc65d23412115bbe40fddec53e1e
Author: BastleyNait <schirinosne@unsa.edu.pe>
Date: Sun Jul 30 12:11:55 2023 -0500

    organizando

commit cab668c10f8c782e0cfca9b863676e4e85b15774
Author: BastleyNait <schirinosne@unsa.edu.pe>
Date: Sun Jul 30 12:11:10 2023 -0500

    implementando modelado

commit 74593d8fbdc0c69f58f5a33800c6a4d76bc723bb
Author: BastleyNait <schirinosne@unsa.edu.pe>
Date: Sun Jul 30 12:09:48 2023 -0500

    implementando metodos 2

commit ee7029c29d0d2dd03edbbac4a81e5319fc0ba029
Author: BastleyNait <schirinosne@unsa.edu.pe>
Date: Sun Jul 30 12:09:33 2023 -0500

    implementando metodos
:...skipping...
commit 3d75087c69b2698e6ea9afb4537051d81f4e67fd (HEAD)
Author: BastleyNait <schirinosne@unsa.edu.pe>
Date: Sun Jul 30 21:07:38 2023 -0500

    implementacion graph stream

commit 42e5d60a4a1adc65d23412115bbe40fddec53e1e
Author: BastleyNait <schirinosne@unsa.edu.pe>
Date: Sun Jul 30 12:11:55 2023 -0500

    organizando

commit cab668c10f8c782e0cfca9b863676e4e85b15774
Author: BastleyNait <schirinosne@unsa.edu.pe>
Date: Sun Jul 30 12:11:10 2023 -0500
```

3.7. Instrucciones para la ejecucion

Listing 5: EJECUCION.txt

1 WINDOWS
2

```
3 javac -cp
   ".;flatlaf-3.1.1.jar;flatlaf-intellij-themes-3.1.1.jar;gs-core-2.0.jar;gs-ui-swing-2.0.jar"
   *.java
4 java -cp
   ".;flatlaf-3.1.1.jar;flatlaf-intellij-themes-3.1.1.jar;gs-core-2.0.jar;gs-ui-swing-2.0.jar"
   TextCompressor
5
6 LINUX
7
8 javac -cp
   ".:flatlaf-3.1.1.jar;flatlaf-intellij-themes-3.1.1.jar;gs-core-2.0.jar;gs-ui-swing-2.0.jar"
   *.java
9 java -cp
   ".:flatlaf-3.1.1.jar;flatlaf-intellij-themes-3.1.1.jar;gs-core-2.0.jar;gs-ui-swing-2.0.jar"
   TextCompressor
```

3.8. Estructura de laboratorio 08

- El contenido que se entrega en este laboratorio es el siguiente:

```
lab08/
| .gitignore
| HuffmanTree.java
| TextCompressor.java
| Leaf.form
| NodoHuff.java
\---latex
| EDA_lab08.pdf
| EDA_lab08.tex
|
+---img
| Commits.png
| logo_abet.png
| logo_episunsa.png
| logo_unsa.jpg
| 1.jpg
| 2.jpg
| 3.jpg
\---src
| HuffmanTree.java
| TextCompressor.java
| Leaf.form
| NodoHuff.java
```

4. Rúbricas

4.1. Entregable Informe

Tabla 1: Tipo de Informe

Informe	
Latex	El informe está en formato PDF desde Latex, con un formato limpio (buena presentación) y fácil de leer.

4.2. Rúbrica para el contenido del Informe y demostración

- El alumno debe marcar o dejar en blanco en celdas de la columna **Checklist** si cumple con el ítem correspondiente.
- Si un alumno supera la fecha de entrega, su calificación será sobre la nota mínima aprobada, siempre y cuando cumpla con todos los ítems.
- El alumno debe autocalificarse en la columna **Estudiante** de acuerdo a la siguiente tabla:

Tabla 2: Niveles de desempeño

Puntos	Nivel			
	Insatisfactorio 25 %	En Proceso 50 %	Satisfactorio 75 %	Sobresaliente 100 %
2.0	0.5	1.0	1.5	2.0
4.0	1.0	2.0	3.0	4.0

Tabla 3: Rúbrica para contenido del Informe y demostración

Contenido y demostración		Puntos	Checklist	Estudiante	Profesor
1. GitHub	Hay enlace URL activo del directorio para el laboratorio hacia su repositorio GitHub con código fuente terminado y fácil de revisar.	2	X	2	
2. Commits	Hay capturas de pantalla de los commits más importantes con sus explicaciones detalladas. (El profesor puede preguntar para refrendar calificación).	4	x	4	
3. Código fuente	Hay porciones de código fuente importantes con numeración y explicaciones detalladas de sus funciones.	2	X	2	
4. Ejecución	Se incluyen ejecuciones/pruebas del código fuente explicadas gradualmente.	2	X	2	
5. Pregunta	Se responde con completitud a la pregunta formulada en la tarea. (El profesor puede preguntar para refrendar calificación).	2	X	2	
6. Fechas	Las fechas de modificación del código fuente estan dentro de los plazos de fecha de entrega establecidos.	2	X	2	
7. Ortografía	El documento no muestra errores ortográficos.	2	X	2	
8. Madurez	El Informe muestra de manera general una evolución de la madurez del código fuente, explicaciones puntuales pero precisas y un acabado impecable. (El profesor puede preguntar para refrendar calificación).	4	x	4	
Total		20		20	

5. Referencias

- <https://www.geeksforgeeks.org/introduction-to-avl-tree/>
- <https://www.geeksforgeeks.org/insertion-in-an-avl-tree/>
- <https://www.geeksforgeeks.org/deletion-in-an-avl-tree/>
- <https://docs.oracle.com/javase/tutorial/java/generics/types.html>
- <https://algorithmtutor.com/Data-Structures/Tree/AVL-Trees/>